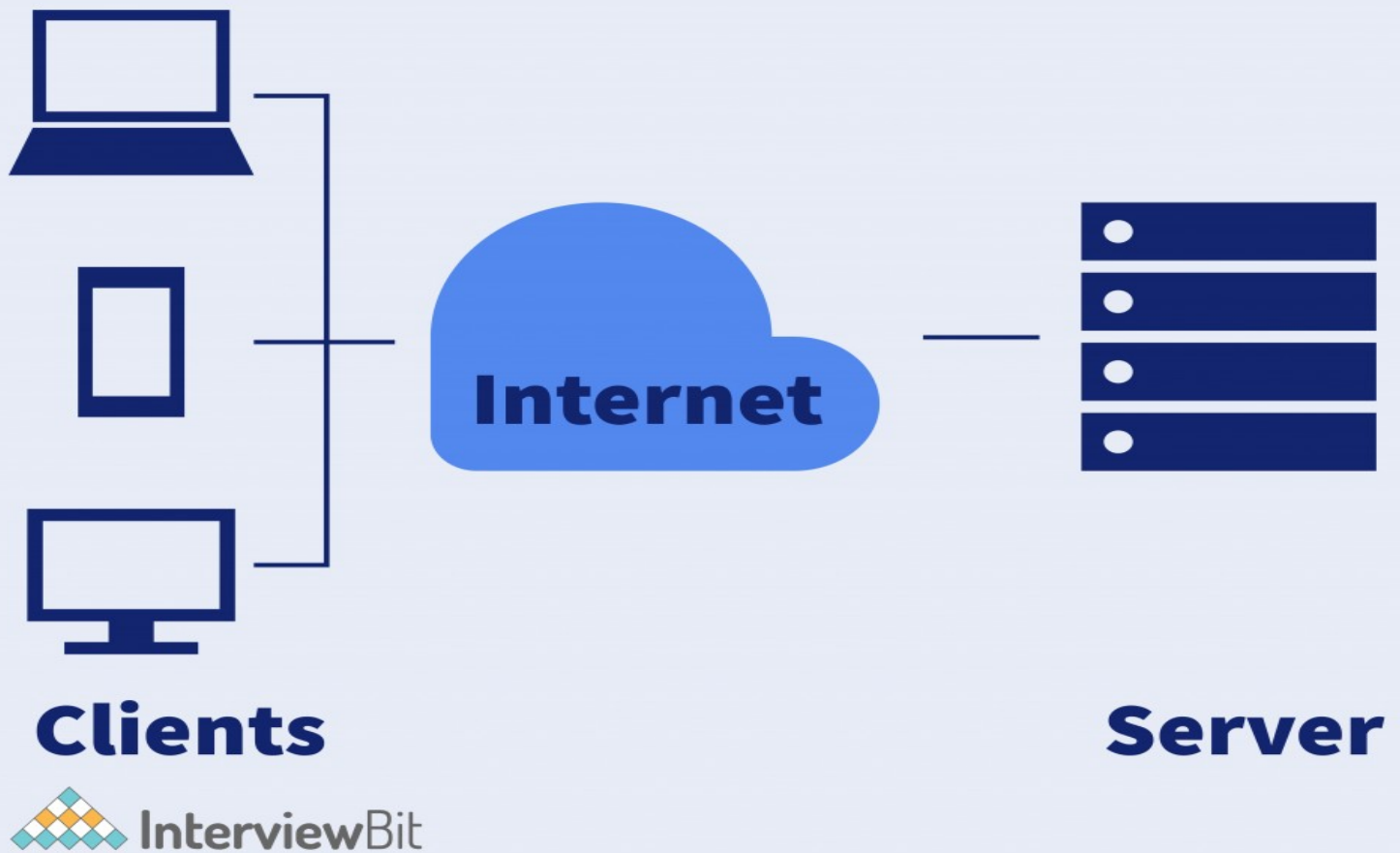




Projet de programmation système : Conception et Réalisation D'une Application de chat client/server



Encadrer par: M .Assane
Ilboudo

Réaliser par: Yameogo
Maimounata(Monique)
Niveau : L3S6

Table des matières

Introduction.....	3
I. Cahier des Charges et Description du Sujet.....	4
1. Présentation générale du projet.....	4
2. Fonctionnalités attendues par version.....	4
Version 1 : Connexion client/serveur et échange simple de messages.....	4
Version 2 : Support multi-client.....	4
Version 3 : Gestion des utilisateurs.....	4
Version 4 : Application de chat complète.....	4
Version 5 : Transfert de fichiers avec approbation.....	4
II. Analyse et Conception de la Solution.....	5
1. Modèle Client/Serveur.....	5
2. Schéma client/serveur.....	5
3. Technologies utilisées.....	5
4. Structures de données.....	5
5. Structure Transfert de Fichier.....	6
6. Description des commandes.....	6
III. Réalisation du Projet.....	7
1. Environnement et outils utilisés.....	7
2. Structure des fichiers source.....	8
3. Description du serveur.....	8
4. Description du client.....	8
IV. Fonctionnalités Implémentées.....	9
Version 1 : Connexion et échange simple.....	9
Version 2 : Serveur Multi-clients.....	9
Version 3 : Gestion des utilisateurs.....	9
Version 4 : Application de Chat.....	9
Version 5 : Transfert de fichiers et fonctionnalités avancées.....	9
V. Tests et Démonstration.....	9
1. Scénarios de tests.....	9
2. Résultats attendus vs résultats obtenus.....	10
Capture du Versions 1.....	11
Capture du Version 2.....	11
Capture du Version 3.....	12
Capture du Version 4.....	12
Capture du Version 5.....	13
VI. Difficultés Rencontrées et Solutions Apportées.....	14
1. Gestion multi-client sans blocage.....	14
2. Réception asynchrone côté client.....	14
3. Gestion des salons et cohérence des messages.....	14
4. Transfert de fichiers — synchronisation et fiabilité.....	14
5. Bonnes pratiques générales.....	14
Conclusion.....	15

Introduction

Ce rapport présente la réalisation d'une application de chat client/serveur développée en langage C dans le cadre du cours de Programmation Système (ECU) .

L'objectif est de concevoir une application réseau complète permettant à plusieurs utilisateurs de communiquer en temps réel via des sockets TCP/IP. Ce projet s'inscrit dans une démarche pratique visant à renforcer la compréhension des mécanismes de communication réseau, de la programmation système et de la gestion des processus et des ressources en environnement Linux.

Les fonctionnalités couvertes comprennent :

- La gestion de salons de discussion
- Le transfert de fichiers entre utilisateurs avec validation du récepteur
- La modération (expulsion d'utilisateurs)
- Les outils statistiques et le journal d'activités (logs)

Le développement a été réalisé de manière incrémentale, en respectant cinq versions successives imposées par le sujet. Ce rapport est structuré comme suit : après cette introduction, nous présentons le cahier des charges, puis l'analyse et la conception de la solution, la réalisation, les tests, les difficultés rencontrées et enfin une conclusion avec des perspectives d'amélioration.

I. Cahier des Charges et Description du Sujet

1. Présentation générale du projet

Le projet consiste à développer une application de chat client/serveur en langage C, utilisant les sockets réseau et le protocole TCP pour assurer la communication entre plusieurs utilisateurs. L'application permet à plusieurs clients de se connecter à un serveur central afin d'échanger des messages textuels et des fichiers. Le développement a été réalisé de manière incrémentale, chaque version apportant de nouvelles fonctionnalités tout en conservant celles des versions précédentes.

2. Fonctionnalités attendues par version

Version 1 : Connexion client/serveur et échange simple de messages

La première version met en place un modèle client/serveur fonctionnel. Le client se connecte au serveur via une adresse IPv4 et un numéro de port. Une fois connecté, il peut envoyer une chaîne de caractères que le serveur lui renvoie (mode écho). La connexion est interrompue proprement lorsque le client envoie la commande /quit.

Version 2 : Support multi-client

Cette version étend la précédente en permettant au serveur de gérer jusqu'à 20 connexions simultanées grâce à la fonction select(). Lorsqu'un 21ème client tente de se connecter, la connexion est refusée avec un message d'erreur approprié.

Version 3 : Gestion des utilisateurs

Une gestion des utilisateurs est introduite. Chaque client doit s'identifier avec un pseudo via la commande /nick. Le serveur gère l'unicité des pseudos et permet d'obtenir la liste des utilisateurs (/who) ainsi que des informations détaillées sur un utilisateur (/whois).

Version 4 : Application de chat complète

La version 4 transforme l'application en un véritable système de chat avec : envoi de messages à tous (broadcast), messages privés (unicast), création et gestion de salons de discussion (multicast). Un salon est automatiquement détruit lorsque son dernier occupant le quitte.

Version 5 : Transfert de fichiers avec approbation

La dernière version ajoute le transfert de fichiers entre utilisateurs. Avant tout transfert, le récepteur doit donner son accord (/accept ou /reject). Une confirmation est envoyée aux deux parties à la fin du transfert.

II. Analyse et Conception de la Solution

1. Modèle Client/Serveur

L'application repose sur une architecture client/serveur TCP. Le serveur centralise toutes les communications. Les clients se connectent via une adresse IP et un port, puis interagissent via des commandes textuelles.

2. Schéma client/server

L'architecture client -Serveur

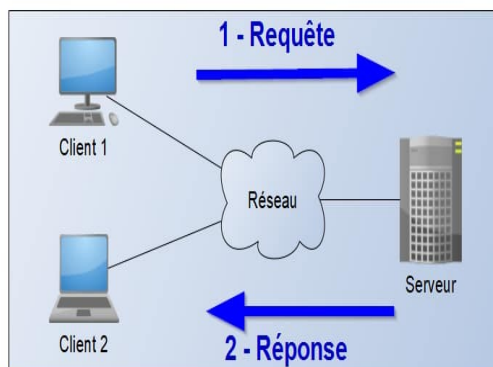


Figure 1: Architecture client/server

3. Technologies utilisées

Élément	Technologie	Rôle
Langage	C (C++)	Développement serveur et client
Protocole	TCP/IP	Communication fiable entre processus
Multiplexage	select()	Gestion multi-clients côté serveur
Threads	POSIX Threads (pthread)	Réception asynchrone côté client
Logs	Fichier chat.log	Traçabilité des événements

4. Structures de données

■ Structure Client

La structure Client représente un utilisateur connecté au serveur. Elle stocke :

- `socket_fd` : le descripteur de socket (-1 = slot libre)
- `pseudo[MAX_PSEUDO_LEN]` : le pseudo choisi avec `/nick`

- salon[MAX_SALON_LEN] : le salon actif (vide si aucun)
- away_msg[BUFFER_SIZE] : message d'absence (vide = disponible)
- nb_messages : nombre de messages envoyés par le client
- Structure Salon

La structure Salon représente un salon de discussion. Elle contient :

- nom[MAX_SALON_LEN] : le nom unique du salon
- password[MAX_PSEUDO_LEN] : mot de passe optionnel
- actif : indicateur d'existence du salon (1 = existant, 0 = slot libre)

5. Structure Transfert de Fichier

La structure FileTransfer gère les échanges de fichiers entre utilisateurs :

- sender_idx : index du client expéditeur
- receiver_idx : index du client destinataire
- filename[MAX_FILENAME] : nom du fichier à transférer
- pending : 1 = en attente de /accept ou /reject

6. Description des commandes

Commande	Description	Type
/nick <pseudo>	Choisir ou changer son pseudo	Identification
/who	Lister les utilisateurs connectés	Information
/whois <pseudo>	Infos détaillées sur un utilisateur	Information
/msg <pseudo> <msg>	Envoyer un message privé	Communication
<message>	Broadcast à tous les utilisateurs	Communication
/create <salon> [mdp]	Créer un salon (mdp optionnel)	Salon
/join <salon> [mdp]	Rejoindre un salon	Salon
/leave	Quitter le salon actuel	Salon
/list	Lister les salons disponibles	Salon
/msgsalon <msg>	Message dans le salon actuel	Salon
/kick <pseudo>	Expulser un utilisateur	Modération
/away [message]	Activer/désactiver un message d'absence	Statut
/stats	Afficher les statistiques du serveur	Serveur
/sendfile <p> <fich>	Demander l'envoi d'un fichier	Fichier
/accept	Accepter un transfert de fichier	Fichier
/reject	Refuser un transfert de fichier	Fichier
/quit	Quitter l'application proprement	Connexion

III. Réalisation du Projet

1. Environnement et outils utilisés

Le projet a été réalisé dans un environnement GNU/Linux (Ubuntu), choisi pour sa stabilité et sa compatibilité native avec la programmation système. Les principaux outils utilisés sont :

- Langage C (C++) : pour l'implémentation du client et du serveur



Figure 2: Langage C

- Compilateur GCC : pour la compilation des programmes



*Figure 3: Compilateur
GCC*

- Visual Studio Code : éditeur de code pour l'écriture des fichiers sources



Figure 4: Visual Studio Code

- Makefile : pour automatiser la compilation du projet



Figure 5: Makefile

2. Structure des fichiers source

Fichier	Description
server.c	Code source du serveur (gestion multi-clients, commandes, logs)
client.c	Code source du client (thread de réception, gestion SIGINT)
Makefile	Script de compilation automatique
chat.log	Fichier de logs généré automatiquement par le serveur
rapport.pdf	Ce rapport de projet

3. Description du serveur

Le serveur constitue le cœur de l'application. Il utilise la fonction `select()` pour surveiller simultanément plusieurs descripteurs de fichiers sans bloquer l'exécution. Il peut gérer jusqu'à 20 clients connectés simultanément. Chaque commande reçue est routée vers sa fonction gestionnaire dédiée (`cmd_nick`, `cmd_who`, `cmd_kick`, etc.), ce qui rend le code modulaire et maintenable.

Le serveur gère : l'acceptation des connexions, la distribution des messages (broadcast, unicast, multicast), la gestion des salons, le transfert de fichiers et la journalisation des événements dans `chat.log`.

4. Description du client

Le client est une application en ligne de commande fonctionnant dans le terminal. Il utilise un thread POSIX pour recevoir les messages en temps réel pendant que le thread principal gère la saisie de l'utilisateur. Une gestion propre du signal SIGINT (Ctrl+C) permet d'envoyer `/quit` au serveur avant la fermeture.

IV. Fonctionnalités Implémentées

Version 1 : Connexion et échange simple

Mise en place de la communication TCP de base : connexion, échange écho, déconnexion propre avec /quit et libération des ressources.

Version 2 : Serveur Multi-clients

Extension du serveur pour gérer 20 connexions simultanées via select(). Refus automatique du 21ème client avec message d'erreur.

Version 3 : Gestion des utilisateurs

Identification obligatoire par pseudo (/nick), unicité garantie, liste des connectés (/who), détails sur un utilisateur (/whois).

Version 4 : Application de Chat

Broadcast (message à tous sauf l'expéditeur), messages privés (/msg), création/gestion/suppression de salons avec mot de passe optionnel, messages de salon (/msgsalon).

Version 5 : Transfert de fichiers et fonctionnalités avancées

Transfert de fichiers avec demande (/sendfile) et validation (/accept ou /reject), confirmation en fin de transfert. Fonctionnalités supplémentaires :

- Modération : expulsion d'utilisateurs (/kick) avec notification
- Message d'absence (/away) avec notification automatique aux correspondants
- Statistiques du serveur (/stats) : uptime, clients actifs, messages, salons
- Horodatage de tous les messages
- Journal d'activités dans chat.log
- SO_REUSEADDR pour redémarrage immédiat du serveur
- Gestion propre de Ctrl+C (SIGINT)

V. Tests et Démonstration

1. Scénarios de tests

- ◆ Connexion de plusieurs clients

Objectif : Vérifier que le serveur accepte jusqu'à 20 clients simultanément et refuse le 21ème.
Résultat : Conforme.

- ◆ Envoi de messages broadcast et privés

Objectif : Vérifier que les messages broadcast sont reçus par tous sauf l'expéditeur, et que les messages privés (/msg) ne sont reçus que par le destinataire. Résultat : Conforme.

- ◆ Création et utilisation des salons

Objectif : Tester la création, le rejoint, les messages dans le salon et la suppression automatique à vide. Résultat : Conforme.

- ◆ Transfert de fichiers avec approbation

Objectif : Vérifier que le fichier n'est transféré qu'après acceptation du récepteur, et qu'un refus (/reject) annule proprement le transfert. Résultat : Conforme.

- ◆ Fonctionnalités avancées

Tests effectués sur /kick, /away, /stats, /list, les logs et l'horodatage. Résultat : Conforme.

2. Résultats attendus vs résultats obtenus

Fonctionnalité	Résultats attendu	Résultat obtenu
Connexion multi_client	Jusqu'à 20 clients acceptés	Conforme
Identification par pseudo	Pseudo unique obligatoire	Conforme
Liste de connectés(/who)	Liste complète	Conforme
Broadcast	Message à tout sauf l'émetteur	Conforme
Message privé(/msg)	Réception par un seul utilisateur	Conforme
Création de salon(/create)	Salon crée ,erreur si doublon	Conforme
Salon avec mot de passe	Accès restreint	Conforme
Message dans salon (/msgsalon)	Visible uniquement par les membres	Conforme
Destruction salon vide	Suppression automatique	Conforme
Transfert de fichier	Approbation+confirmation	Conforme
Refus de transfert(/rejet)	Annulation propre	Conforme
Expulsion(/kick)	Déconnexion +notification	Conforme
Message d'absence(/away)	Notification automatique	Conforme
Statistique(stats)	Uptime,clients ,messages salons	Conforme
Horodatage des messages	Heure affichée sur chaque message	Conforme

Fonctionnalité	Résultats attendu	Résultat obtenu
Journal d'activités(chat.log)	Logs complets	Conforme

Les tests effectués montrent que l’ensemble des fonctionnalités répond correctement aux exigences définies dans le cahiers des charges. Voici les différents captures des cinq versions :

Capture du Versions 1

```
mainounata@mainounata-HP-EliteBook-840-G5: $ ls -l /chat_project/
version1 version2 version3 version4 version5
mainounata@mainounata-HP-EliteBook-840-G5: $ ^[[200-cd /chat_project/version1
cd : commande introuvable
mainounata@mainounata-HP-EliteBook-840-G5: $ cd /chat_project/version1
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$ make
gcc server.c -o server
gcc client.c -o client
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$ ./server
✓ Socket créée
✓ Bind effectué sur le port 8080
✓ Serveur en écoute...
✓ Client connecté !
Client déconnecté.
✓ Sockets fermées.
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$ ./server
✓ Socket créée
✓ Bind effectué sur le port 8080
✓ Serveur en écoute...
✓ Client connecté !
[Client] : bonjour maman
[Client] : /quit
Connexion fermée.
✓ Sockets fermées.
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$
```

```
mainounata@mainounata-HP-EliteBook-840-G5: $ cd /chat_project/version1
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$ ./client 8080 127.0.0.1
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
> bonjour ma beatrice
Connexion fermée.
✓ Socket fermée.
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$ ./client 8080 127.0.0.1
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
> bonjour maman
[Server] : bonjour maman
> /quit
[Server] : You will be terminated

Connection terminated
✓ Socket fermée.
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$
```

Capture du Version 2

```
✓ Serveur en écoute...
✓ Client connecté !
[Client] : bonjour maman
[Client] : /quit
Connexion fermée.
✓ Sockets fermées.
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version1$ cd /chat_project/version2
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version2$ make
gcc server.c -o server
gcc client.c -o client
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version2$ ./server
✓ Socket créée
✓ Bind effectué sur le port 8080
✓ Serveur en écoute (max 20 clients)...
✓ Client connecté (socket 4) 1/20 clients
✓ Client connecté (socket 5) 2/20 clients
[Client 4] : salut
[Client 5] : je suis heureuse
[Client 5] : for i in $(seq 1 21); do
[Client 5] : ./client 8080 127.0.0.1 &
[Client 5] : sleep 0.2

✓ Client connecté (socket 6) 3/20 clients
✓ Client connecté (socket 7) 4/20 clients
✓ Client connecté (socket 8) 5/20 clients
✓ Client connecté (socket 9) 6/20 clients
✓ Client connecté (socket 10) 7/20 clients
✓ Client connecté (socket 11) 8/20 clients
✓ Client connecté (socket 12) 9/20 clients
✓ Client connecté (socket 13) 10/20 clients
✓ Client connecté (socket 14) 11/20 clients
✓ Client connecté (socket 15) 12/20 clients
✓ Client connecté (socket 16) 13/20 clients
✓ Client connecté (socket 17) 14/20 clients
✓ Client connecté (socket 18) 15/20 clients
✓ Client connecté (socket 19) 16/20 clients
✓ Client connecté (socket 20) 17/20 clients
✓ Client connecté (socket 21) 18/20 clients
✓ Client connecté (socket 22) 19/20 clients
✓ Client connecté (socket 23) 20/20 clients
△ Connexion refusée (serveur plein)
△ Connexion refusée (serveur plein)
△ Connexion refusée (serveur plein)
```

```
mainounata@mainounata-HP-EliteBook-840-G5: $ cd /chat_project/version2
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version2$ for i in $(seq 1 21); do
> ^[[200-./client 8080 127.0.0.1 &
> ./client 8080 127.0.0.1 & sleep 0.2 done
> ^C
mainounata@mainounata-HP-EliteBook-840-G5: /chat_project/version2$ cd /chat_project/version1
n2
for i in $(seq 1 21); do
./client 8080 127.0.0.1 &
sleep 0.2
done
[1] 95466
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
[1]+ Arrêté ./client 8080 127.0.0.1
[2] 95468
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
[2]+ Arrêté ./client 8080 127.0.0.1
[3] 95471
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
[3]+ Arrêté ./client 8080 127.0.0.1
[4] 95473
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
[4]+ Arrêté ./client 8080 127.0.0.1
[5] 95475
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
[5]+ Arrêté ./client 8080 127.0.0.1
[6] 95477
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
[6]+ Arrêté ./client 8080 127.0.0.1
[7] 95479
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
>
```

Capture du Version 3

```
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version3$ ./server
✓ Socket créée
Erreur bind: Address already in use
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version3$
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version3$ ./server
✓ Socket créée
✓ Bind effectué sur le port 8080
✓ Serveur en écoute (max 20 clients)...
✓ Client connecté (socket 4) 1/20
[anonyme] : /nick mini
✓ Client connecté (socket 5) 2/20
[anonyme] : /nick charle
[mini] : /who
[mini] : /whois karim
[mini] : /whois charle

mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version3$ ./client 8080 127.0.0.1
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
[Server] : Bienvenue ! Identifiez-vous avec /nick <pseudo>
/nick mini
[Server] : Pseudo changé en 'mini'
/who
[Server] : Utilisateurs connectés :
- mini
- charle
/whois karim
[Server] : Utilisateur 'karim' introuvable.
/whois charle
[Server] : Utilisateur 'charle' connecté sur socket 5
```

Capture du Version 4

```
✓ Socket créée
Erreur bind: Address already in use
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$ ./server
✓ Socket créée
Erreur bind: Address already in use
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$ ./server
✓ Socket créée
✓ Bind effectué sur le port 8080
✓ Serveur en écoute (max 20 clients)...
✓ Client connecté (socket 4) 1/20
[anonyme] : /nick mini
✓ Client connecté (socket 5) 2/20
[anonyme] : /nick charle
[mini] : Bonjour tout le monde !
[mini] : /msg charle voici ton cd
[mini] : /create general
[mini] : /join general
[mini] : /msgalon Bienvenue dans le salon !
[charle] : /join general
[charle] : /msgalon Bonjour depuis le salon !
[charle] : /leave
[mini] : /list
[mini] : /quit

mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$ cd ~/chat_project/version4
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$ ./client 8080 127.0.0.1
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
[Server] : Bienvenue ! Identifiez-vous avec /nick <pseudo>
/nick mini
[Server] : Pseudo changé en 'mini'
Bonjour tout le monde !
/msg charle voici ton cd
[Server] : [Privé à charle] : voici ton cd
/create general
[Server] : Salon 'general' créé.
/join general
[Server] : Vous avez rejoint le salon 'general'.
/msgalon Bienvenue dans le salon !
[Server] : [Salon general][charle] : Bonjour depuis le salon !
/list
[Server] : Salons disponibles :
- general
/quit
[Server] : You will be terminated
Connection terminated
✓ Socket fermée.
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$

mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$ cd ~/chat_project/version4
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$ ./client 8080 127.0.0.1
✓ Socket créée
✓ Connecté au serveur 127.0.0.1:8080
[Server] : Bienvenue ! Identifiez-vous avec /nick <pseudo>
/nick charle
[Server] : Pseudo changé en 'charle'
[Server] : [mini] : Bonjour tout le monde !
[Server] : [Privé de mini] : voici ton cd
/join general
[Server] : Vous avez rejoint le salon 'general'.
/msgalon Bonjour depuis le salon !
/leave
[Server] : Vous avez quitté le salon 'general'.
/quit
[Server] : You will be terminated
Connection terminated
✓ Socket fermée.
mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/version4$
```

mainounata@mainounata-HP-EliteBook-840-G5:~/chat_project/...

Capture du Version 5

```
[mini] : /list
[cherie] : /quit
[mini] : /quit
^C
maimounata@maimounata-HP-EliteBook-840-G5: /chat_project/version$ cd -/chat_project/versio
n5
maimounata@maimounata-HP-EliteBook-840-G5: /chat_project/version$ make
gcc server.c -o server
gcc client.c -o client -lpthread
maimounata@maimounata-HP-EliteBook-840-G5: /chat_project/version$ ./server
✓ Socket créée
✓ Bind effectué sur le port 8080
✓ Serveur en écoute (max 20 clients)...
✓ Nouveau client (socket 4) - 1/20
[anonyme] : /nick mai
✓ Nouveau client (socket 5) - 2/20
[anonyme] : /nick boubu
[mai] : /sendfile boubu rapport.txt
[boubu] : /accept
[mai] : /file Voici le contenu du fichier rapport
[mai] : /sendfile boubu test.txt
[boubu] : /reject
[mai] : /away
[mai] : /kick boubu
```

```
> /sendfile boubu rapport.txt
Demande envoyée à 'boubu'. En attente...

FILE_SEND rapport.txt

'boubu' a accepté. Envoyez le contenu avec //file <contenu>

> /file Voici le contenu du fichier rapport
Fichier 'rapport.txt' envoyé avec succès à 'boubu'.

> /sendfile boubu test.txt
Demande envoyée à 'boubu'. En attente...

'boubu' a refusé 'test.txt'.

> /away
Vous êtes maintenant disponible.

> /kick boubu
*** boubu a été expulsé du serveur ***

'boubu' a été expulsé avec succès.

> []
```

```
Tapez /quit pour quitter

Bienvenue ! Identifiez-vous avec /nick <pseudo>
Tapez /help pour voir les commandes.

> /nick boubu
Pseudo changé en 'boubu'

[mai] veut vous envoyer 'rapport.txt'. Tapez /accept ou /reject.

> /accept
Vous avez accepté 'rapport.txt'.

FICHIER REQU de 'mai' : Voici le contenu du fichier rapport

[mai] veut vous envoyer 'test.txt'. Tapez /accept ou /reject.

> /reject
Vous avez refusé 'test.txt'.

Vous avez été expulsé du serveur par un modérateur.

> [serveur] Connexion fermée par le serveur
```

maimounata@maimounata-HP-EliteBook-840-G5: /chat_project/...

VI. Difficultés Rencontrées et Solutions Apportées

1. Gestion multi-client sans blocage

Difficulté : Permettre au serveur de communiquer avec plusieurs clients simultanément sans bloquer l'exécution, tout en respectant la limite de 20 connexions.

Solution : Utilisation de `select()` pour surveiller plusieurs sockets simultanément. Un tableau de structures `Client` stocke l'état de chaque connexion. Un contrôle systématique est effectué à chaque nouvelle connexion pour refuser les clients au-delà de la limite.

2. Réception asynchrone côté client

Difficulté : Le client doit pouvoir saisir des messages et recevoir des messages du serveur en même temps, sans que l'un bloque l'autre.

Solution : Un thread POSIX dédié à la réception des messages est lancé au démarrage du client. Le thread principal gère la saisie, tandis que le thread de réception affiche les messages entrants en temps réel.

3. Gestion des salons et cohérence des messages

Difficulté : Limiter la diffusion des messages aux seuls membres d'un salon, gérer les cas d'erreur (salon inexistant, doublon) et supprimer automatiquement les salons vides.

Solution : Chaque salon contient un champ actif et une clé salon stockée dans la structure `Client`. Lors de l'envoi d'un message dans un salon, le serveur vérifie l'appartenance avant diffusion. Un mécanisme de comptage des membres déclenche la suppression automatique quand un salon devient vide.

4. Transfert de fichiers — synchronisation et fiabilité

Difficulté : Synchroniser l'émetteur et le récepteur, gérer l'approbation avant transfert, envoyer des données sans corruption et confirmer le succès des deux côtés.

Solution : La structure `FileTransfer` maintient l'état du transfert (pending). Le serveur vérifie l'existence du fichier côté émetteur avant d'envoyer la demande au récepteur. Le transfert n'est déclenché qu'après /accept explicite. Les données sont envoyées par blocs, et une confirmation finale est adressée aux deux parties.

5. Bonnes pratiques générales

- Vérification systématique des valeurs de retour des appels système (`socket`, `bind`, `listen`, `recv`, `send`)
- Libération correcte des ressources à chaque déconnexion ou fermeture
- Utilisation de `SO_REUSEADDR` pour éviter l'erreur 'Address already in use' au redémarrage
- Tests progressifs après chaque version pour valider le fonctionnement avant d'ajouter de nouvelles fonctionnalités

Conclusion

L'application réalisée permet la connexion simultanée de plusieurs clients, l'échange de messages en mode broadcast et privé, la gestion des utilisateurs par pseudos, la création et l'utilisation de salons de discussion, le transfert de fichiers avec mécanisme de confirmation, ainsi que des fonctionnalités avancées comme la modération, les statistiques et le journal d'activités.

Les objectifs fixés dans le cahier des charges ont été atteints. Le système développé est fonctionnel et conforme aux attentes. Cependant, certaines limites peuvent être relevées :

- Interface uniquement en mode terminal (pas d'interface graphique)
- Absence de mécanismes de sécurité avancés (authentification, chiffrement des échanges)
- Performances limitées lors du transfert de fichiers volumineux

Des améliorations futures peuvent être envisagées :

- Développement d'une interface graphique plus conviviale
- Intégration du chiffrement des communications (par exemple avec OpenSSL)
- Optimisation du transfert de fichiers (envoi par blocs plus grands, reprise après interruption)
- Persistance des données (historique des messages, comptes utilisateurs)
- Amélioration des performances globales du serveur pour un plus grand nombre de clients

Ces perspectives montrent que ce projet constitue une base solide pour le développement d'applications réseau plus complètes, robustes et sécurisées.